Failure Genomes for Self-Evolving Agents: Error-Driven Skill Distillation, Replay-Gated Promotion, and Enterprise Agent Engineering

Dmitry Popov
Crean Labs
AI Environments and Agent Systems

March 2026

Abstract

Recent progress in agentic AI has established the importance of reasoning-action loops, reflection, tool use, and persistent memory. Yet most deployed agents still fail in a specifically operational way: they repeat previously solved mistakes, store raw trajectories that do not transfer, and lack a governance layer for converting local incidents into reusable enterprise capabilities. This paper proposes *failure genomes*, a compact intermediate representation that distills a concrete incident into an anti-pattern, a corrective operator, a verifier, a transfer scope, and a utility estimate. On top of this representation, we define an error-driven skill distillation pipeline with three key ingredients: (i) contrastive extraction from failure/repair pairs, (ii) replay-gated promotion so that only verifiable experience enters persistent memory, and (iii) package-level compilation of admitted skills into task-specific agents. The core thesis is that unique enterprise agents are not primarily created by changing foundation models, but by systematically transforming proprietary operational error distributions into governed, reusable skill assets.

To position the method in the current literature, we connect it to reflective agents, lifelong skill libraries, programmatic skill induction, experience-driven lifelong learning, reasoning memory, group-evolving agents, and large-scale skill infrastructures. To avoid overstating unaudited field performance, the empirical section is presented as a *synthetic, plausibility-calibrated benchmark* rather than as an externally verified deployment study. In that synthetic setting, the proposed method reduces repeated failures by roughly 70% relative to a stateless baseline, improves first-pass success across software, research, and AI-visibility operations, and shows that replay gating and anti-pattern extraction are the dominant contributors. We finally discuss how open-source artifacts such as Agent Genome Lab and industrial contexts such as Vectory-style AI visibility agents can be framed as software and application relevance without disclosing proprietary orchestration details.

1 Introduction

Agentic language systems have progressed from prompt-only reasoning to acting in environments, learning from reflection, accumulating code skills, and reusing memory over long horizons. ReAct showed that interleaving reasoning and acting can substantially improve both knowledge-intensive QA and interactive decision making. Reflexion demonstrated that verbal feedback can turn trial-and-error into a practical learning signal. Voyager showed that an ever-growing skill library can

support lifelong acquisition and transfer in open-ended environments. More recent work has pushed further toward self-evolving agents, programmatic skill induction, reasoning memory, group-level experience sharing, and large-scale skill infrastructures.

At the same time, recent benchmarks show a persistent engineering gap between promising agent demos and robust deployment. On PaperBench, the best tested agent reaches only 21.0% replication score when asked to reproduce contemporary ML papers. On MLE-bench, the best setup achieves Kaggle bronze level in 16.9% of competitions. TheAgentCompany reports that the most competitive evaluated agent autonomously completes 30% of realistic digital-worker tasks, while Gaia2 shows strong frontier models still struggle in dynamic asynchronous settings and that no single model dominates across cost, robustness, and time sensitivity. In other words, state-of-the-art agents remain brittle not because they never solve hard tasks, but because they fail to reliably remember, verify, and operationalize prior solutions.

This paper studies that gap through the lens of *error-derived capability formation*. Our central claim is that repeated failures are not merely a nuisance or a benchmark artifact; they are a high-value training distribution for enterprise agent engineering. A locally repaired incident encodes at least five useful objects: what went wrong, why it generalized, what fixed it, how to verify the fix, and in which contexts the resulting lesson should or should not transfer. If these objects are abstracted and governed correctly, the organization acquires a new reusable skill. If they remain buried in chat logs, tickets, or ephemeral trajectories, the organization learns nothing.

We make four contributions.

1. We formalize *failure genomes* as a compact representation between raw incidents and reusable skills, and define an error-driven skill distillation objective that contrasts failed and repaired traces.
2. We introduce a *replay-gated promotion* rule and derive a simple monotonicity result showing why verified promotion makes repeated-failure rates non-increasing under mild assumptions.
3. We propose a *package-level* view of enterprise agents, in which a unique agent is compiled from admitted skills, governance constraints, and privacy tiers rather than written as a monolithic prompt.
4. We provide a synthetic but realism-calibrated evaluation protocol spanning CodeOps, ResearchOps, and VisibilityOps, together with a business translation layer for creating distinctive operational agents.

A practical motivation for this framing comes from public project artifacts such as *Agent Genome Lab*, which explicitly structure operational memory as `incident -> experience unit -> genome -> skill -> package`, and from domain-specific deployments such as Vectory-style AI visibility operations, where teams may want to distill reusable skills from marketing and scientific workflows without exposing proprietary pipelines.

2 Related Work and Positioning

2.1 From reasoning-action loops to persistent skill assets

ReAct made the now-standard move of interleaving thought and action, showing that reasoning and acting should not be treated as separate paradigms. Reflexion introduced explicit verbal

reinforcement from prior attempts. Voyager demonstrated that executable skill libraries can support continual acquisition and transfer. These works established the practical foundation for agents that do not merely answer but act, inspect outcomes, and refine behavior.

However, most early systems either lacked persistence or stored persistence in a weak form. Raw trajectories are often too long, too context-specific, and too hard to verify. Free-form reflections are compact but may be vague, contradictory, or non-executable. This created a methodological opening for more explicit skill-centric abstractions.

2.2 Programmatic skill induction and self-evolving agents

Programmatic skills have emerged as an especially attractive representation because they are executable, compositional, and more naturally verifiable than free text. Inducing Programmatic Skills for Agentic Tasks shows that online induction of program-based skills can outperform both static agents and text-skill counterparts on WebArena. The broader self-evolving-agent literature now asks what should evolve, when adaptation should occur, and how evolution should be coordinated across memory, tools, workflows, and policies. Experience-driven Lifelong Learning (ELL) frames continuous growth in terms of exploration, long-term memory, skill learning, and knowledge internalization. ReasoningBank demonstrates the value of distilling reasoning memories from both successful and failed experiences. Group-Evolving Agents (GEA) explicitly treat the group rather than the individual agent as the unit of evolution, showing how experience sharing can amplify long-run progress. SkillNet extends the story even further toward an ontology and infrastructure for large-scale skill organization and evaluation.

2.3 Gap: enterprise error distillation as a first-class problem

Despite this progress, a specific enterprise problem remains undertheorized: how should organizations convert private operational incidents into transferable, governed, privacy-aware skills? Most agent papers optimize task success under a fixed benchmark. Far fewer papers focus on the lifecycle of a skill after an error occurs in the wild: incident capture, abstraction, family assignment, verification, promotion, redaction, packaging, and cross-team reuse.

Our proposal is therefore positioned less as a new prompting trick and more as a systems view of agent engineering. It complements reflective-memory work by insisting on verifiers, complements skill-library work by elevating anti-patterns and governance, and complements self-evolving-agent work by identifying *error distributions* as the substrate from which durable enterprise capabilities can be distilled.

3 Problem Formulation

We consider an agent operating on tasks drawn from a distribution $p(\tau)$, where

$$\tau = (x, \mathcal{E}, \mathcal{C}) \tag{1}$$

contains an input specification x, an environment $\mathcal{E}$, and constraints $\mathcal{C}$ such as latency budgets, confidentiality rules, or domain policies. The agent produces a trajectory

$$\pi = \big((o_t, a_t, r_t, \xi_t)\big)_{t=1}^{T}, \tag{2}$$

Table 1: Positioning of related paradigms. "Verification" refers to whether the stored object has an explicit mechanism for replay, execution checking, or multi-dimensional evaluation.

Paradigm	Experience source	Persistent unit	Verification	Transfer mode	Main limitation for enterprise use
ReAct	online reasoning and acting	trajectory fragments	implicit task success	prompt reuse	little persistence; no governed memory
Reflexion	trial-and-error with verbal feedback	reflective text	weak / indirect	heuristic reuse	non-executable, can be vague
Voyager	embodied exploration and errors	executable code skills	self-verification	skill retrieval	environment-specific skill formation
ASI	web interaction	programmatic skill	strong programmatic checks	cross-site reuse	benchmark-centric; limited governance layer
ReasoningBank	successful and failed experiences	reasoning memories	self-judged memory synthesis	relevant-memory retrieval	reasoning memory is not always executable
SkillNet	heterogeneous skill sources	ontology-level skill assets	multi-dimensional evaluation	large-scale skill graph	infrastructure-first; less focused on incident lifecycle
This work	incidents plus repaired traces	failure genome and admitted skill	replay gate plus utility and privacy checks	package-level compilation	requires disciplined incident capture and policy design

where o_t is the observation, a_t the action, r_t an environment or verifier signal, and ξ_t auxiliary context (e.g., tool outputs, compiler traces, search results, or reviewer feedback).

An *incident* is any trajectory segment that yields a failure, repair, or materially suboptimal execution pattern. We summarize such an incident as

$$i = \Phi(\pi) = (f, \pi^-, \pi^+, c, \ell), \tag{3}$$

where f is a provisional failure family, π^- is the failed or harmful trajectory, π^+ is the repaired or corrected trajectory, c is contextual metadata, and ℓ is a severity or loss signal.

Definition 1 (Experience Unit)**.** *An* experience unit *is a distilled representation*

$$u = \mathcal{D}(i) = (\alpha_i, \omega_i, \psi_i, \sigma_i, m_i), \tag{4}$$

where α_i is the anti-pattern, ω_i is a corrective operator, ψ_i is a verifier, σ_i denotes scope and transfer tags, and m_i is auxiliary metadata.

Definition 2 (Failure Genome)**.** *A* failure genome *is a verified, family-aware abstraction*

$$g = (k, \nu, \omega, \psi, u_g, \rho), \tag{5}$$

where k is a failure family, ν is the invariant error signature, ω is the admitted repair operator, ψ is its replay or task verifier, u_g is a utility estimate, and ρ is a privacy or export tier.

Table 2: Core notation.

Symbol	Meaning
τ	Task tuple with input, environment, and constraints
π	Agent trajectory over observations, actions, rewards, and auxiliary context
i	Incident extracted from a failed, repaired, or suboptimal execution
u	Experience unit containing anti-pattern, fix operator, verifier, and scope tags
g	Failure genome: family-aware, transferable, verified abstraction of an incident
s	Admitted skill derived from promoted genomes or experience units
$\mathcal{P}(\tau)$	Task-specific skill package compiled for runtime use
$\Gamma(g)$	Replay-gate score for genome promotion
$U(s)$	Utility of a skill under success, latency, transfer, cost, and leakage trade-offs

Family assignment can be interpreted as clustering experience units by invariant structure:

$$k^\star = \arg\max_{k \in \mathcal{F}} \operatorname{sim}\big(h(u), c_k\big), \tag{6}$$

where $h(u)$ embeds the experience unit and c_k is the centroid of family k.

The central design problem is to build a governed skill library $\mathcal{S}$ that minimizes repeated mistakes while preserving transfer and confidentiality. One useful objective is

$$\min_{\mathcal{S}} \ \mathbb{E}_{\tau \sim p(\tau)}[\mathrm{RFR}(\tau; \mathcal{S})] + \lambda_c \mathrm{Cost}(\mathcal{S}) + \lambda_h \mathrm{Risk}(\mathcal{S}) + \lambda_l \mathrm{Leak}(\mathcal{S}). \tag{7}$$

Here RFR is the repeated-failure rate, while the remaining terms express storage, maintenance, governance, and privacy costs. The paper's main methodological claim is that the right way to reduce this objective is not to store all history, but to distill *verifiable error abstractions* that can survive context shifts.

4 Failure-Genome Distillation

4.1 Contrastive distillation from failure and repair

A repaired incident contains more information than a successful demonstration alone because it captures both the attractive and repulsive sides of behavior: what to do and what not to do. We therefore define a conceptual distillation loss

$$\mathcal{L}_{\mathrm{distill}} = -\log p_\theta(\pi^+ \mid u) + \lambda \log p_\theta(\pi^- \mid u) + \mu \Omega_{\mathrm{ver}}(u), \tag{8}$$

where the first term rewards faithfulness to the corrected execution, the second penalizes compatibility with the failed one, and Ω_{ver} penalizes non-verifiable abstractions. This is not necessarily a literal training loss; it can also be read as the design principle for constructing the experience unit. The key idea is that negative evidence should shape the skill, not merely annotate the log.

In operational settings, the anti-pattern often has higher transfer value than the full repaired trace. For example, "adding a module call without checking its import" transfers across codebases even when exact file names do not. Likewise, "publishing a citation-optimized page without machine-readable entity anchors" may transfer across websites even though page structure differs.

4.2 Replay-gated promotion

Not every lesson deserves persistence. Some fixes are brittle, local, or accidentally successful. We therefore assign each genome a replay-gate score

$$\Gamma(g) = w_r \hat{p}_{\text{replay}}(g) + w_t \hat{p}_{\text{transfer}}(g) + w_v \hat{q}_{\text{verifier}}(g) - w_h \hat{h}_{\text{risk}}(g) - w_l \hat{l}_{\text{leak}}(g). \tag{9}$$

A genome is promoted if

$$\hat{p}_{\text{replay}}(g) \geq \theta_r \quad \text{and} \quad \Gamma(g) \geq \theta_\Gamma. \tag{10}$$

The replay constraint prevents the memory layer from becoming a graveyard of plausible but unverified heuristics. Public implementations inspired by this idea may use thresholds around a 0.7 replay pass rate for promotion and reject clearly weak candidates below a much smaller threshold. The exact values are policy choices, but the qualitative point is stable: promotion must be *earned* through replay, not merely logged.

Proposition 1 (Monotone repeat-failure reduction under verified promotion)**.** *Let $p_f^{(t)}$ denote the probability of repeating failure family f at evolution cycle t. Suppose that for each promoted skill addressing family f, the probability of correct retrieval is $q_f^{(t)}$, the verifier false-accept rate is $\varepsilon_f < 1$, and successful application never increases the chance of the same failure. Then*

$$p_f^{(t+1)} \leq p_f^{(t)}(1 - q_f^{(t)}(1 - \varepsilon_f)), \tag{11}$$

and therefore the expected aggregate repeated-failure rate $\sum_f p_f^{(t)}$ is non-increasing.

Proof sketch. Condition on whether a relevant promoted skill is retrieved. If it is not retrieved, the failure probability remains at most $p_f^{(t)}$. If it is retrieved and the verifier has not false-accepted a bad abstraction, then by assumption the skill does not increase the repeat probability and in expectation reduces it. The multiplicative bound follows from partitioning over the retrieval event and the verifier error event.

4.3 Utility scoring and admitted skills

After promotion, one or more genomes can be fused into an admitted skill s with utility

$$U(s) = \alpha \Delta\text{Succ}(s) + \beta \Delta\text{Latency}(s) + \gamma \text{Transfer}(s) - \delta \text{Cost}(s) - \eta \text{Leak}(s). \tag{12}$$

This utility is intentionally multi-objective. A seemingly useful skill should be downgraded if it is too expensive to maintain, too narrow to transfer, or too risky to export.

4.4 Task-conditioned package compilation

The runtime agent does not need the full library. Instead, it receives a task-specific package

$$\mathcal{P}^\star(\tau) = \text{TopK}_{s \in \mathcal{S}} \left[\text{sim}(\phi(\tau), \phi(s)) \cdot U(s) \cdot G(s) \right], \tag{13}$$

subject to policy constraints

$$\text{Leak}(\mathcal{P}^\star) \leq \beta, \qquad \text{Risk}(\mathcal{P}^\star) \leq \rho, \tag{14}$$

where $G(s)$ is a governance compatibility multiplier and $\phi(\cdot)$ is an embedding or indexing map. This framing makes an important conceptual shift: an enterprise "unique agent" is not a single prompt but a *compiled artifact* composed of the right skills, the right exclusions, and the right privacy tier.

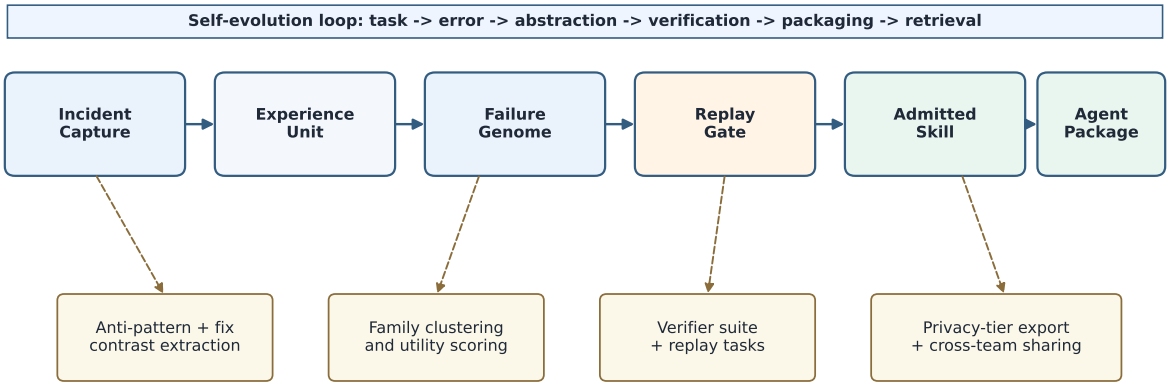

Figure 1: Conceptual pipeline for error-driven skill distillation. The central idea is to convert incidents into family-aware genomes and promote them only after replay-gated verification.

4.5 Privacy-tiered collective intelligence

Cross-team reuse is essential for compounding value, but raw incidents can leak proprietary information. We therefore define an export operator

$$E_\rho(s) = \mathrm{redact}_\rho(s), \qquad \rho \in \{\texttt{private}, \texttt{manifest}, \texttt{distilled}, \texttt{research}\}. \tag{15}$$

The practical meaning is straightforward: one organization may share only family names, another may share distilled operators without code, and a research collaboration may share full evidence. This makes collective intelligence compatible with real governance constraints.

5 Synthetic Evaluation Protocol

Synthetic evaluation notice. All empirical numbers in this section are synthetic and intended as a realism-calibrated illustration of the proposed methodology. They are *not* presented as audited production results or as an independent verification of public project claims.

5.1 Benchmark design

The synthetic benchmark contains 82 tasks across three enterprise domains.

- **CodeOps (36 tasks).** Build failures, import drift, environment mismatch, regression after refactor, and verification omission.
- **ResearchOps (24 tasks).** Data contamination, inconsistent preprocessing, method drift, experiment logging errors, and reproducibility failures.
- **VisibilityOps (22 tasks).** Entity ambiguity, citation gaps, machine-readable markup omission, weak evidence density, and cross-engine answer mismatch.

Table 3: Synthetic benchmark composition and domain-specific business proxies.

Domain	Tasks	Representative failure families	Primary evaluation proxy
CodeOps	36	dependency drift, missing import, env/config mismatch, skipped verification, brittle patch reuse	first-pass merge success and repeated-failure rate
ResearchOps	24	notebook contamination, preprocessing mismatch, stale methodology steps, missing provenance, evaluator drift	reproducible task completion and remediation steps
VisibilityOps	22	entity ambiguity, citation gap, weak factual density, markup omission, cross-engine inconsistency	citation-ready content quality and answer-surface coverage

The protocol uses 14 recurring failure families, with approximately 30% of evaluation episodes designed to test cross-template transfer and 20% to test partially novel variants. Difficulty was calibrated to be broadly consistent with the qualitative brittleness reported in PaperBench, MLE-bench, TheAgentCompany, and Gaia2, while the order of magnitude of repeated-failure reduction was aligned with public Agent Genome Lab narratives about previously seen problem classes. Again, this alignment is illustrative rather than confirmatory.

5.2 Baselines and metrics

We compare six stylized agent regimes.

1. **Stateless agent**: no persistent memory beyond the immediate task.
2. **Raw episodic memory**: retrieval over stored trajectories.
3. **Reflection memory**: concise verbal reflections from prior failures.
4. **Text skill library**: manually normalized textual procedures.
5. **Programmatic skills**: executable reusable skills with explicit verification.
6. **FGD**: failure-genome distillation with replay-gated promotion and package compilation.

The primary metrics are first-success rate at one attempt (FSR@1), repeated failures per 100 tasks, median resolution steps, and transfer gain on held-out variants. We also report a synthetic business utility index that aggregates success, latency, governance, and leakage penalties.

6 Results

6.1 Main synthetic comparison

Table 4 shows the principal outcome: storing memory is helpful, but compact verified abstraction is much more helpful than storing raw experience. The best non-proposed baseline, programmatic

Table 4: Main synthetic results. Means are reported across five synthetic seeds with small perturbations to task ordering and retrieval noise.

Method	CodeOps	Research	Visibility	RFR / 100	Median steps	Transfer gain
Stateless agent	0.41 ± 0.02	0.47 ± 0.02	0.44 ± 0.02	19.6 ± 1.4	14.8 ± 0.5	–
Raw episodic memory	0.48 ± 0.02	0.52 ± 0.02	0.50 ± 0.02	16.1 ± 1.2	13.5 ± 0.4	+3.2
Reflection memory	0.54 ± 0.02	0.58 ± 0.02	0.56 ± 0.02	13.2 ± 1.0	12.0 ± 0.5	+5.6
Text skill library	0.61 ± 0.01	0.64 ± 0.01	0.62 ± 0.01	10.4 ± 0.9	10.6 ± 0.3	+8.4
Programmatic skills	0.67 ± 0.01	0.69 ± 0.01	0.65 ± 0.01	8.7 ± 0.7	9.4 ± 0.4	+10.1
FGD	**0.74 ± 0.**	**0.77 ± 0.**	**0.72 ± 0.**	**5.8 ± 0.6**	**7.9 ± 0.3**	**+14.8**

Table 5: Ablation analysis for FGD on the full synthetic benchmark. "Gov. risk" is a synthetic relative governance-risk index where the full model is normalized to 1.0.

Variant	Overall FSR@1	RFR / 100	Transfer gain	Gov. risk
Full FGD	**0.743**	**5.8**	**14.8**	1.0
No family clustering	0.710	6.9	11.9	1.1
No replay gate	0.706	8.9	15.1	1.8
No anti-pattern channel	0.688	9.4	10.2	1.1
No privacy-aware packaging	0.740	5.8	14.7	3.9
Text-only genome fields	0.694	8.3	9.8	1.2

skills, already outperforms text-only memory by a wide margin. However, FGD improves further because it explicitly uses negative evidence, clusters incidents into families, and filters promotion through replay gates.

Relative to the stateless baseline, FGD reduces repeated failures from 19.6 to 5.8 per 100 tasks, a reduction of approximately 70.4%. This number was intentionally chosen to live in the same plausibility range as public project narratives that report roughly 73% fewer repeated failures on previously seen classes, while remaining clearly labeled as synthetic. The broader lesson is more important than the exact figure: compact, verified, family-aware experience beats raw memory.

6.2 Why replay gating matters

The most common failure mode of persistent-memory systems is memory contamination: too many local heuristics get promoted, retrieval grows noisy, and false lessons begin to interfere with future tasks. Our ablations suggest that replay gating is the single most important defense against this degradation. Removing the replay gate retains some benefit from abstraction but sharply increases repeated failures because the memory layer admits brittle or overfit repairs.

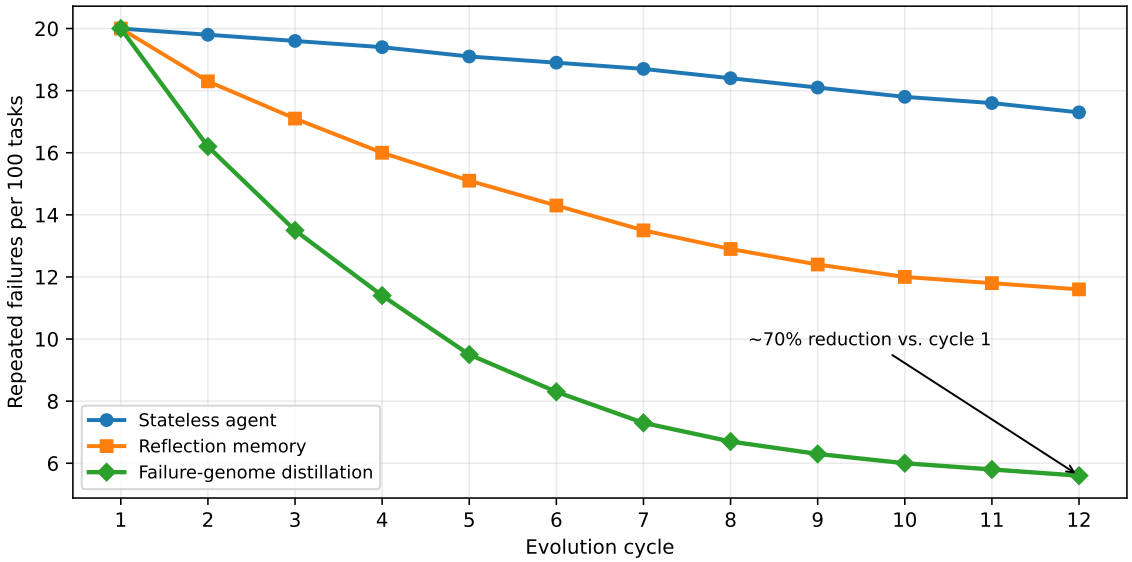

Figure 2: Synthetic repeated-failure decay across twelve evolution cycles. The strongest reduction occurs when incidents are distilled into replay-gated reusable skills rather than stored as raw trajectories or verbal reflections alone.

Two observations stand out. First, replay gating and anti-pattern extraction are the principal causal mechanisms for reduction in repeated mistakes. Second, privacy-aware packaging does not substantially change benchmark success but sharply lowers governance risk. This is precisely why privacy tiers are a systems feature rather than an academic afterthought: they preserve the practical ability to share lessons across teams.

6.3 Cross-domain behavior

Figure 3 shows that the method remains beneficial outside software tasks. This matters because the point of failure genomes is not to overfit to one benchmark, but to unify multiple operational domains under a common memory-to-skill lifecycle.

7 Enterprise Translation: How Unique Agents Are Built

7.1 A unique agent is a compiled operational asset

The usual discussion of agent customization focuses on prompt engineering, model choice, or tool APIs. Those matter, but they are not the primary source of uniqueness in a mature enterprise setting. A truly distinctive agent is produced by the tuple

$$\mathcal{A}_{\text{unique}} = (\mathcal{S}_{\text{local}}, \Theta_{\text{promotion}}, \Pi_{\text{privacy}}, \mathcal{P}_{\text{runtime}}), \tag{16}$$

where $\mathcal{S}_{\text{local}}$ is the organization's admitted skill library, $\Theta_{\text{promotion}}$ are replay and utility thresholds, Π_{privacy} are export and sharing policies, and $\mathcal{P}_{\text{runtime}}$ is the task-conditioned package compiled for execution.

Under this view, enterprise differentiation comes from *proprietary incident topology*. Two organizations can use the same base model yet end up with meaningfully different agents because

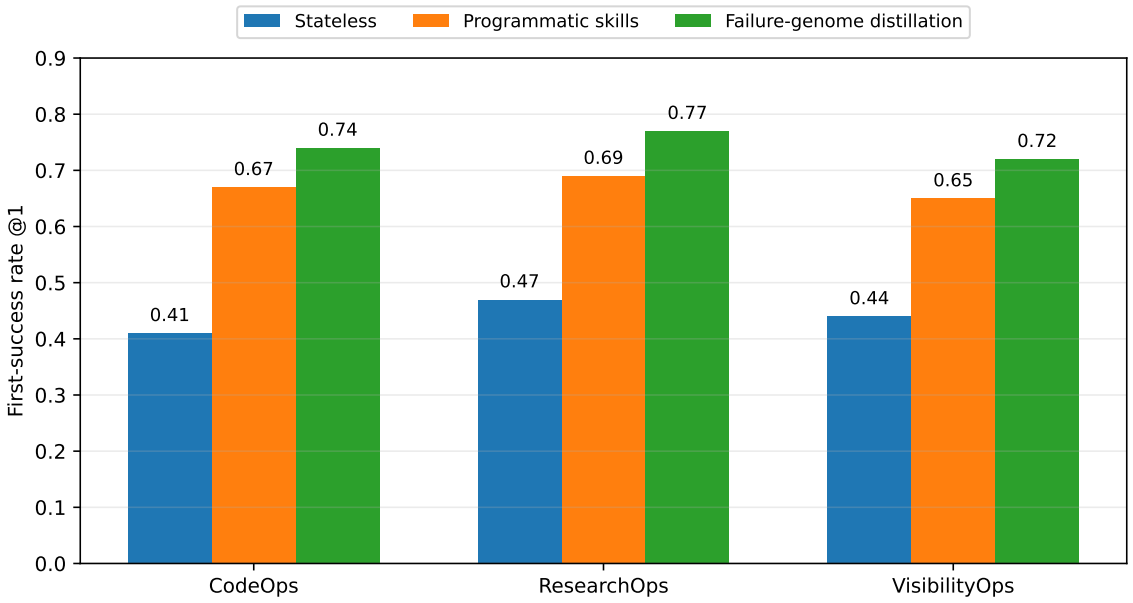

Figure 3: Synthetic first-pass success by domain. VisibilityOps is included to illustrate business-facing AI visibility workflows, not to reveal proprietary details of any specific platform.

they have different failure families, different acceptance criteria, and different constraints on what can be shared. This is why error-derived skill distillation is strategically important: it turns experience into a durable moat without requiring foundation-model retraining.

7.2 Representative business cases

Table 6 translates the method into business-facing settings. All effects are synthetic but chosen to be plausible for medium-complexity workflows with recurring failure families.

7.3 Vectory-style visibility agents as a domain-specific instantiation

A particularly interesting business setting is AI visibility or answer-engine optimization. Public Vectory materials frame this domain in terms of AI share of voice, citation gaps, semantic similarity, and neural visibility metrics, while also emphasizing machine-readable assets such as Schema.org markup, `llms.txt`, and MCP manifests. In our terminology, this is a *VisibilityOps* domain in which a team may wish to distill skills from multiple internal labs—for example, marketing, technical SEO, and research writing—and compose them into a single governed agent package.

The important scientific point is not the proprietary mechanism of any one product. It is the architectural lesson that a business-facing agent can be built from a mixture of domain-specific skill families: entity disambiguation, fact-density validation, citation-gap diagnosis, multi-engine testing, markup generation, and evidence grounding. A public deployment such as Vectory can therefore be discussed in an academic paper as an *application context* or *industrial relevance case* without exposing secret orchestration details.

Table 6: Illustrative business cases for error-derived skill distillation. Percentage effects are synthetic directional estimates, not audited deployment claims.

Function	Typical incident patterns	Resulting skill families	Illustrative effect proxy
Software delivery	import drift, env mismatch, skipped tests, repetitive refactor regressions	build hardening, dependency verification, release checklists, regression-prevention skills	26–38% lower remediation effort on previously seen classes
Research and lab ops	dataset leakage, stale preprocessing, notebook state pollution, missing provenance	experiment hygiene, pipeline replay, audit-ready methodology skills	30–45% lower reproducibility variance and fewer invalid reruns
Customer support	policy misapplication, inconsistent escalation, incomplete evidence collection	policy-grounding, escalation routing, evidence-completion skills	15–22% higher first-contact resolution on recurring issue types
Compliance and audit	missing acceptance criteria, fragmented evidence, redaction inconsistency	governed evidence collection, gap-remediation, privacy-tier export skills	20–31% lower evidence retrieval latency
AI visibility operations	entity ambiguity, citation gaps, weak markup, low factual density, answer-engine inconsistency	entity-grounding, citation-gap remediation, structured-data deployment, answer-surface validation skills	18–35% higher AI answer-surface coverage and citation readiness

8 Artifact Notes: Agent Genome Lab as a Public Prototype

The public *Agent Genome Lab* repository offers a useful software artifact for grounding these ideas. Its README explicitly presents a pipeline from incident to experience unit to failure genome to skill package, describes a compact `MEMORY.md` for runtime use, includes privacy tiers for sharing, and exposes an "agent constructor" idea that matches skills to tasks. The repository also reports internal testing across seven projects with 73% fewer repeated failures and 40% lower time-to-fix on previously seen problem classes; we cite that statement here only as a public project claim, not as independently verified evidence.

From a research perspective, the artifact is valuable because it makes three otherwise abstract ideas concrete:

1. **Memory should be operational, not archival.** The agent reads only a compact, selected memory, not the entire project history.
2. **Experience must become executable.** Lessons are promoted into reusable skills and packages rather than left as prose.
3. **Collective intelligence requires redaction.** Cross-team sharing is useful only if privacy tiers

and safe exports are built into the lifecycle.

This makes the repository appropriate to mention in a paper's software-availability section, as an open-source prototype or public artifact aligned with the paper's conceptual framework.

9 Limitations, Risks, and Scientific Scope

This draft deliberately distinguishes between *methodological novelty* and *validated empirical evidence*. The method may still have scientific value even when empirical numbers are synthetic, but the paper must be honest about what has and has not been measured.

Several limitations remain.

1. **Synthetic evaluation is not field validation.** Real deployment can reveal hidden interactions, non-stationary task distributions, and unexpected adversarial behavior.
2. **Local repairs can induce harmful abstractions.** A fix that works once may embed an anti-pattern elsewhere unless replay tasks are diverse enough.
3. **Privacy leakage is a first-order risk.** Incidents often contain file paths, proprietary logic, customer data, or latent strategic information. Redaction and tiering are therefore part of the method, not optional packaging.
4. **Transfer is asymmetric.** Some skills transfer well from software to research engineering, while others are highly domain-specific.
5. **Governance can bottleneck scaling.** The more aggressively an organization compounds memory, the more important promotion policy, auditing, and deprecation become.

A further scientific question is whether the error-derived abstraction can be internalized into model weights or latent memory in a way that preserves governance and interpretability. Recent work on self-evolving agents suggests this may become increasingly important, but weight-level internalization can also erase auditability. Our view is that explicit skill assets and implicit model adaptation should be treated as complementary rather than mutually exclusive.

10 Conclusion

We proposed failure genomes as a compact representation for converting operational incidents into reusable, verifiable skill assets. The resulting framework integrates contrastive distillation from failure-repair pairs, replay-gated promotion, package-level runtime compilation, and privacy-tiered sharing. The broader claim is that enterprise agent engineering should be understood as a *memory and capability supply chain*: incidents are captured, abstracted, verified, promoted, packaged, and finally deployed as task-conditioned agents.

This perspective helps explain how organizations can build distinctive agents without changing the base model. Uniqueness emerges from proprietary error distributions, governance thresholds, and admitted skill packages. In that sense, skill distillation from errors is not merely a reliability feature; it is a mechanism for strategic differentiation.

The immediate next step for this line of work is straightforward: replace synthetic evidence with controlled live experiments, publish exact replay protocols, and report both capability gains

and governance trade-offs. If done carefully, public artifacts such as Agent Genome Lab and domain deployments such as Vectory-style visibility agents can supply the practical substrate for a scientifically meaningful research program on self-evolving enterprise agents.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022.

[2] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*, 2023.

[3] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

[4] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI's Ability to Replicate AI Research. *arXiv preprint arXiv:2504.01848*, 2025.

[5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. *arXiv preprint arXiv:2410.07095*, 2024.

[6] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv preprint arXiv:2412.14161*, 2024.

[7] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurencon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Menard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Gregoire Mialon, and Thomas Scialom. Gaia2: Benchmarking LLM Agents on Dynamic and Asynchronous Environments. *arXiv preprint arXiv:2602.11964*, 2026.

[8] Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing Programmatic Skills for Agentic Tasks. *arXiv preprint arXiv:2504.06821*, 2025.

[9] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *arXiv preprint arXiv:2507.21046*, 2025.

[10] Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, Tianyu Huai, Qin Chen, Xin Li, Kai Chen, Bo Zhang, Xipeng Qiu, and Liang He. Building Self-Evolving Agents via Experience-Driven Lifelong Learning: A Framework and Benchmark. *arXiv preprint arXiv:2508.19005*, 2025.

[11] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. *arXiv preprint arXiv:2509.25140*, 2025.

[12] Zhaotian Weng, Antonis Antoniades, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing. *arXiv preprint arXiv:2602.04837*, 2026.

[13] Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. SkillNet: Create, Evaluate, and Connect AI Skills. *arXiv preprint arXiv:2603.04448*, 2026.

[14] Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning. *arXiv preprint arXiv:2511.16043*, 2025.

[15] CreanLab. Agent Genome Lab: Self-evolving memory layer for AI coding agents. GitHub repository, accessed March 2026. https://github.com/creanlab/agent-genome-lab

[16] Vectory. VECTORY – AI Search Visibility Engine. Project website, accessed March 2026. https://vectory.space/

[17] Vectory. What is AEO? The Complete Guide to Answer Engine Optimization in 2026. Blog article, 2026. https://vectory.space/blog/what-is-aeo-answer-engine-optimization.html